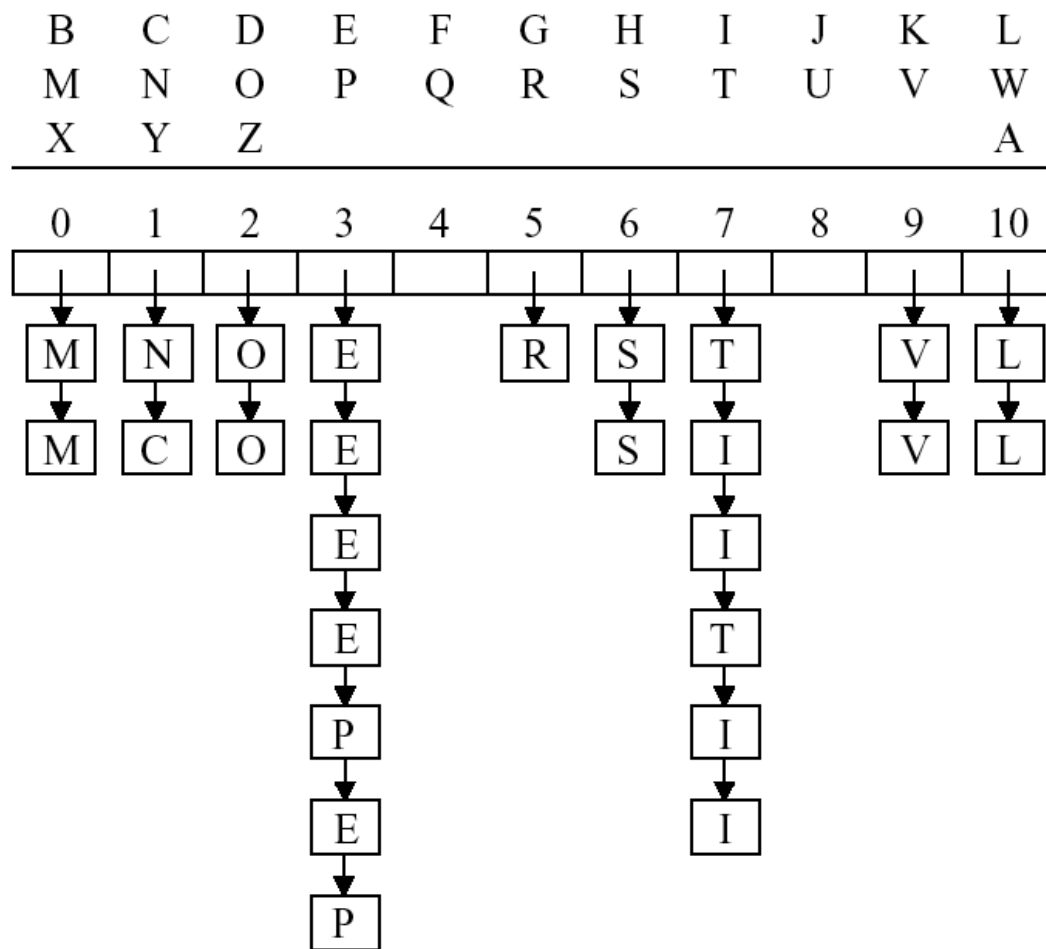




Hash table: implementazione

Martedì 31 marzo

Liste di collisione (Separate chaining)



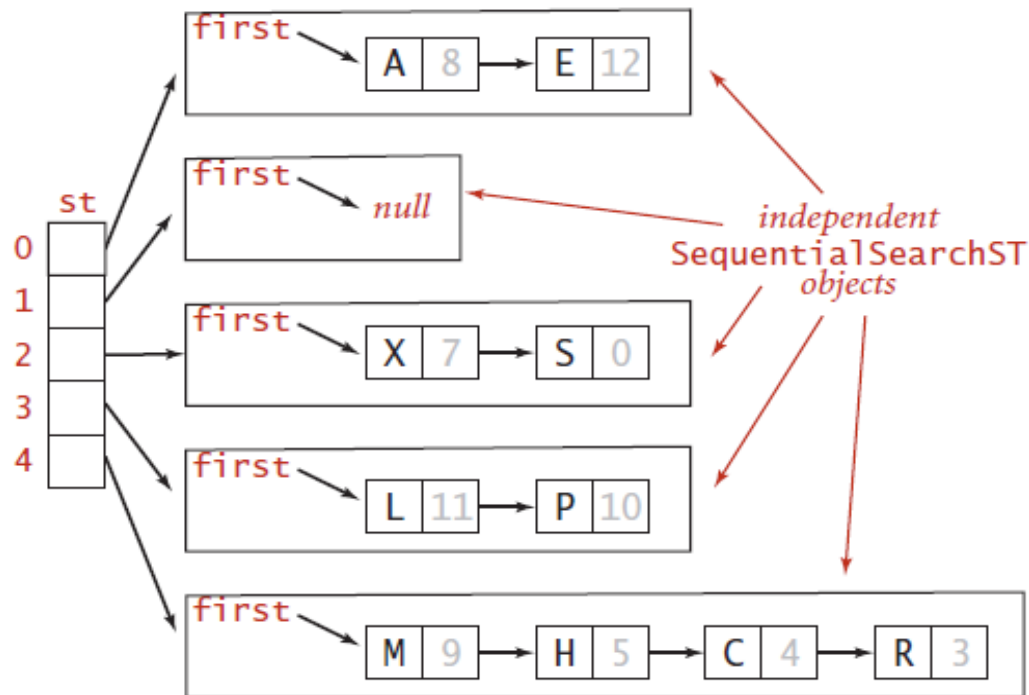
Esempio di tabella hash basata su liste di collisione contenente le lettere della parola:

**PRECIPITEVOLISSI
MEVOLMENTE**

Esempio

key hash value

S	2	0
E	0	1
A	0	2
R	4	3
C	4	4
H	4	5
E	0	6
X	2	7
A	0	8
M	4	9
P	3	10
L	3	11
E	0	12



Nel dettaglio

- + Separate chaining per la gestione delle collisioni
 - + Array di $M < N$ liste concatenate.
 - + Hash: da chiave k a intero i compreso tra 0 e $M - 1$;
 - + Insert: metti in testa alla i -esima lista (se non già presente);
 - + Search: ricerca lungo la i -esima lista.

Operazioni di dizionario

classe `TavolaHashListeColl` **implementa** `Dizionario`:

dati:

$$S(m, n) = \Theta(m + n)$$

un array v di dimensione m in cui ogni cella contiene un puntatore a una lista di coppie $(elem, chiave)$. Un elemento e con chiave $k \in U$ è nel dizionario se e solo se (e, k) è nella lista puntata da $v[h(k)]$, con $h : U \rightarrow \{0, \dots, m-1\}$ funzione hash con uniformità semplice calcolabile in tempo $O(1)$.

operazioni:

`insert(elem e, chiave k)`

$$T(n) = O(1)$$

aggiungi la coppia (e, k) alla lista puntata da $v[h(k)]$.

`delete(chiave k)`

$$T_{avg}(n) = O(1 + n/m)$$

rimuovi la coppia (e, k) nella lista puntata da $v[h(k)]$.

`search(chiave k)  $\rightarrow$  elem`

$$T_{avg}(n) = O(1 + n/m)$$

se (e, k) è nella lista puntata da $v[h(k)]$, allora restituisci e , altrimenti restituisci `null`.

Implementazione in Java

- +

```
public Value get (Key key ) {  
    int i = hash ( key );  
    for (Node x = s t [ i ]; x != null ; x = x . next )  
        i f ( key . equals ( x . key ) ) return ( Value ) x . val ;  
    return null ;  
}
```
- +

```
public void put (Key key , Value val ) {  
    int i = hash ( key );  
    for (Node x = s t [ i ]; x != null ; x = x . next )  
        i f ( key . equals ( x . key ) ) { x . val = val ; return ; }  
    s t [ i ] = new Node( key , val , s t [ i ] );  
}
```
- + Nota: in SeparateChainingHashST.java si usa un array private `SequentialSearchST<Key, Value>[] st` che contiene già questi metodi;

Implementazione in Java

```
public class SeparateChainingHashST<Key, Value>
{
    private int N; // number of key-value pairs
    private int M; // hash table size
    private SequentialSearchST<Key, Value>[] st; // array of ST objects
    public SeparateChainingHashST() { this(997); }
    public SeparateChainingHashST(int M)
    { // Create M linked lists.
        this.M = M;
        st = (SequentialSearchST<Key, Value>[]) new SequentialSearchST[M];
        for (int i = 0; i < M; i++)
            st[i] = new SequentialSearchST();
    }
    private int hash(Key key)
    { return (key.hashCode() & 0x7fffffff) % M; }

    public Value get(Key key)
    { return (Value) st[hash(key)].get(key); }

    public void put(Key key, Value val)
    { st[hash(key)].put(key, val); }

}
```

Resize

- + Una soluzione più robusta consiste nell'utilizzare il resize dell'array per essere sicuri che le liste siano corte indipendentemente dal numero di coppie di valori-chiave presenti nella tabella

Separate chaining: proprietà

- + Il numero di confronti per ricerca e inserimento è circa N/M ;
- + M troppo grande $\rightarrow$ troppe liste vuote;
- + M troppo piccolo $\rightarrow$ liste troppo lunghe;
- + Scelta tipica: $M = N/5 \rightarrow$ operazioni a tempo costante.
 - + Si può fare array resizing quando ci si discosta troppo da questo valore.

Indirizzamento aperto

Supponiamo di voler inserire un elemento con chiave k e la sua posizione "naturale" $h(k)$ sia già occupata.

L'indirizzamento aperto consiste nell'occupare un'altra cella, anche se potrebbe spettare di diritto a un'altra chiave.

Cerchiamo la cella vuota (se c'è) ispezionando altre celle secondo una sequenza di indici che dipende dalla chiave:

Deve essere una permutazione di
 $(0, 1, \dots, M-1)$

$c(k, 0), c(k, 1), c(k, 2), \dots, c(k, m-1)$

Pseudo codice

classe TavolaHashAperta **implementa** Dizionario:

dati: $S(m) = \Theta(m)$
un array v di dimensione m in cui ogni cella contiene una coppia $(elem, chiave)$.

operazioni:

insert($elem\ e, chiave\ k$)

1. **for** $i = 0$ **to** $m - 1$ **do**
2. **if** $(v[c(k, i)].elem = \text{null})$ **then**
3. $v[c(k, i)] \leftarrow (e, k)$
4. **return**
5. **errore** tavola piena

Vedremo in seguito la cancellazione

search($chiave\ k$) $\rightarrow elem$

1. **for** $i = 0$ **to** $m - 1$ **do**
2. **if** $(v[c(k, i)].elem = \text{null})$ **then**
3. **return** null
4. **if** $(v[c(k, i)].chiave = k)$ **then**
5. **return** $v[c(k, i)].elem$
6. **return** null

Metodi di ispezione: scansione lineare

$$c(k,i) = (h(k) + i) \bmod m$$

per $0 \leq i < m$

Esempio

	C E I L M N O P R S T V																														
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
P																			P												
R																			P		R										
E								E											P		R										
C						C		E											P		R										
I						C		E				I							P		R										
P						C		E				I							P	P	R										
I						C		E				I	I						P	P	R										
T						C		E				I	I						P	P	R										
E						C		E	E			I	I						P	P	R										
V						C		E	E			I	I						P	P	R										
O						C		E	E			I	I						O	P	P	R									
L						C		E	E			I	I						O	P	P	R									
I						C		E	E			I	I	I	L				O	P	P	R									
S						C		E	E			I	I	I	L				O	P	P	R									
S						C		E	E			I	I	I	L				O	P	P	R									
I						C		E	E			I	I	I	L	I			O	P	P	R									
M						C		E	E			I	I	I	L	I	M		O	P	P	R									
E						C		E	E	E		I	I	I	L	I	M		O	P	P	R									
V						C		E	E	E		I	I	I	L	I	M		O	P	P	R									
O						C		E	E	E		I	I	I	L	I	M		O	P	P	R									
L						C		E	E	E		I	I	I	L	I	M		O	P	P	R									
M						C		E	E	E		I	I	I	L	I	M		O	P	P	R									
E						C		E	E	E	E	I	I	I	L	I	M		O	P	P	R									
N						C		E	E	E	E	I	I	I	L	I	M		O	P	P	R									
T						C		E	E	E	E	I	I	I	L	I	M		O	P	P	R									
E	E					C		E	E	E	E	I	I	I	L	I	M		O	P	P	R									

Inserimenti in tabella hash basata su indirizzamento aperto con scansione lineare delle lettere della parola:

**PRECIPITEVOLISSI
MEVOLMENTE**

4,8 celle scandite in media per inserimento

PIÙ IN DETTAGLIO

key	hash	value	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
S	6	0							S									
E	10	1							0				E					
A	4	2					A		S				E					
R	14	3					2		0				1				R	
C	5	4					A	C	S				E				R	
H	4	5					2	5	0				1				3	
E	10	6					A	C	S	H			E				R	
X	15	7					2	5	0	5			6				3	X
A	4	8					A	C	S	H			E				R	X
M	1	9		M			8	5	0	5			6				3	7
P	14	10	P	M			8	5	0	5			6				3	7
L	6	11	P	M			8	5	0	5			6				3	7
E	10	12	P	M			8	5	0	5	11		E				3	7

entries in red are new

entries in gray are untouched

keys in black are probes

probe sequence wraps to 0

keys[]

vals[]

Metodi di scansione: hashing doppio

La scansione lineare o quadratica provocano effetti di agglomerazione, cioè lunghi gruppi di celle consecutive occupate che rallentano la scansione

L'hashing doppio riduce il problema

$$c(k,i) = \lfloor h_1(k) + i \cdot h_2(k) \rfloor \bmod m$$

per $0 \leq i < m$, h_1 e h_2 funzioni hash

	C		E		I		L		M		N		O		P		R		S		T		V								
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
P																		P													
R																		P		R											
E							E											P		R											
C						C	E											P		R											
I						C	E				I							P		R											
P						C	P	E			I							P		R											
I						C	P	E			I							P		R			I								
T						C	P	E			I							P		R		T	I								
E						C	P	E			I				E			P		R		T	I								
V						C	P	E			I				E			P		R		T	I	V							
O						C	P	E			I				E		O	P		R		T	I	V							
L						C	P	E			I			L	E		O	P		R		T	I	V							
I						I	C	P	E		I			L	E		O	P		R		T	I	V							
S						I	C	P	E		I			L	E		O	P		R	S	T	I	V							
S						I	C	P	E		I	S		L	E		O	P		R	S	T	I	V							
I						I	C	P	E		I	S		L	E	I	O	P		R	S	T	I	V							
M	M					I	C	P	E		I	S		L	E	I	O	P		R	S	T	I	V							
E	M					I	C	P	E	E		I	S		L	E	I	O	P		R	S	T	I	V						
V	M					I	C	P	E	E		I	S		L	E	I	O	P		R	S	T	I	V	V					
O	M					I	C	P	E	E	O		I	S		L	E	I	O	P		R	S	T	I	V	V				
L	M					I	C	P	E	E	O		I	S		L	E	I	O	P		R	S	T	I	V	V				L
M	M	M				I	C	P	E	E	O		I	S		L	E	I	O	P		R	S	T	I	V	V				L
E	M	M	E			I	C	P	E	E	O		I	S		L	E	I	O	P		R	S	T	I	V	V				L
N	M	M	E			I	C	P	E	E	O		I	S		L	E	I	O	P	N	R	S	T	I	V	V				L
T	M	M	E			I	C	P	E	E	O		I	S	T	L	E	I	O	P	N	R	S	T	I	V	V				L
E	M	M	E			I	C	P	E	E	O	E	I	S	T	L	E	I	O	P	N	R	S	T	I	V	V				L
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30

Inserimenti in tabella hash basata su indirizzamento aperto con hashing doppio delle lettere della parola:

**PRECIPITEVOLISSI
MEVOLMENTE**

3,1 celle scandite in media per inserimento

Analisi del costo di scansione

Nessuna di queste tecniche soddisfa l'ipotesi di hashing uniforme (la scansione lineare genera $\Theta(M)$ sequenze di ispezioni differenti, l'hashing doppio $\Theta(M^2)$).

Si dimostra che il tempo richiesto in media da un'operazione di ricerca di una chiave, assumendo che le chiavi siano prese con probabilità uniforme da U :

<i>esito ricerca</i>	<i>sc. lineare</i>	<i>hashing doppio</i>
chiave trovata	$\frac{1}{2} + \frac{1}{2(1-\alpha)}$	$-\frac{1}{\alpha} \log_e(1 - \alpha)$
chiave non trovata	$\frac{1}{2} + \frac{1}{2(1-\alpha)^2}$	$\frac{1}{1-\alpha}$

LINEAR PROBING: Implementazione

```
public class LinearProbingHashST<Key, Value>
{
    private int N;
    // number of key-value pairs in the table
    private int M = 16;
    // size of linear-probing table
    private Key[] keys; // the keys
    private Value[] vals; // the values

    public LinearProbingHashST()
    {
        keys = (Key[]) new Object[M];
        vals = (Value[]) new Object[M];
    }

    private int hash(Key key)
    { return (key.hashCode() & 0x7fffffff) % M; }

    private void resize() //resize dell'array

    public void put(Key key, Value val)
    {
        if (N >= M/2) resize(2*M); // double M
        int i;
        for (i = hash(key); keys[i] != null; i = (i + 1) % M)
            if (keys[i].equals(key)) { vals[i] = val; return; }
        keys[i] = key;
        vals[i] = val;
        N++;
    }

    public Value get(Key key)
    {
        for (int i = hash(key); keys[i] != null; i = (i + 1) % M)
            if (keys[i].equals(key))
                return vals[i];
        return null;
    }
}
```

Linear probing: cancellazione

- + La cancellazione può provocare buchi che rendono alcune chiavi irraggiungibili (es., cancella C e poi cerca H nel tracing).
- + Dobbiamo re-inserire (re-hashing) tutte le chiavi del cluster alla destra del buco.

Implementazione

```
public void delete (Key key ) {  
    if ( ! contains ( key ) ) return ;  
    int i = hash ( key ) ;  
  
    while ( ! key . equals ( keys [ i ] ) )  
        i = ( i + 1 ) % m ;  
    keys [ i ] = null ; vals [ i ] = null ;  
    i = ( i + 1 ) % m ;  
  
    while ( keys [ i ] != null ) {  
        Key keyToRehash = keys [ i ] ;  
        Value valToRehash = vals [ i ] ;  
        keys [ i ] = null ; vals [ i ] = null ;  
        n-- ;  
        put ( keyToRehash , valToRehash ) ;  
        i = ( i + 1 ) % m ;  
    }  
    n-- ;  
    if ( n > 0 && n <= m / 8 ) resize ( m / 2 ) ;  
}
```

Array resizing

- + Fattore di carico $\alpha = N/M < 1$;
per il linear probing il fattore di carico ha un significato diverso che per il separate chaining (dove assume un valore maggiore di 1). Qui rappresenta la percentuale delle entries della tabella che sono occupate, è importante che non raggiunga il valore 1 (che significa tabella piena!)
- + Il ridimensionamento saltuario della tabella è necessario per evitare la condizione di tabella piena (infinite loop), ed è consigliabile per tenere piccola la dimensione dei cluster;
- + I cluster lunghi hanno maggior probabilità di crescere in taglia a seguito di un probe.
- + E' opportuno che sia $1/8 < \alpha < 1/2$.

Array resizing

```
private void resize ( int capacity ) {  
    LinearProbingHashST <Key , Value> temp = new LinearProbingHashST <Key , Value >( capacity );  
    for ( int i = 0; i < m; i ++ ) {  
        if ( keys [ i ] != null ) {  
            temp . put ( keys [ i ] , vals [ i ] );  
        }  
    }  
    keys = temp . keys ;  
    vals = temp . vals ;  
    m = temp . m ;  
}
```

Esercizio

- + Usando l'implementazione [SeparateChainingHashST.java](#), creare una tabella hash con chiavi intere, in cui vengono inseriti N valori casuali, con $N=10, 10^2, 10^3, 10^4, 10^5$, and 10^6 . Per ogni valore di N , valutare quante liste vuote sono presenti nella tabella dopo l'inserimento degli N elementi.
- + Modificare `SeparateChainingHashST` per usare una seconda funzione hash e prendere la più corta delle due liste. Dare una traccia del processo di inserimento delle chiavi `EASYQUESTION` nell'ordine in una tabella inizialmente vuota di taglia $M=3$ usando la funzione $11k \% M$ (per la k -esima lettera dell'alfabeto) come prima funzione hash e la funzione $17k \% M$ per la k -esima lettera dell'alfabeto come seconda funzione hash. Realizzare l'inserimento di $N=100$ chiavi casuali nell'alfabeto delle lettere maiuscole ed effettuare un plot del numero di confronti effettuati per ciascun inserimento.
- + Aggiungere un metodo `put_dupl` che inserisce chiavi con duplicazione.